CPUSim – A Pipelined CPU & Memory Hierarchy Simulator

Project X 2026 – Task 2

Submitted By

Darshan Mahale

Language

C++17

Institute

Veermata Jijabai Technological Institute (VJTI)

Branch

IT

GitHub Repository

https://github.com/d4rshnn/CPUSim-ProjectX2026

Submission Components

- Source Code
- Input Files
- Stress Test Cases
- Documentation Report
- Execution Screenshots

Note on AI Usage

AI tools were used during this project for learning concepts, discussing ideas, debugging issues, and improving the presentation of this report.

However, the overall implementation, testing, and refinement of the simulator were carried out through my own step-by-step development process. The project involved understanding the requirements, writing code, validating outputs, fixing bugs, and making improvements through multiple iterations before reaching the final solution.

1. Introduction

This project implements a CPU scheduling and memory hierarchy simulator in C++. The objective is to model how tasks compete for CPU execution time and how memory blocks move through a cache hierarchy consisting of L1, L2, L3 caches and RAM.

The simulator combines scheduling decisions with memory access behavior to visualize cache hits, cache misses, memory latency, FIFO eviction, and overall execution statistics.

The simulator was not built as a single program from the beginning. Instead, the problem was divided into smaller phases so that each component could be understood and implemented separately. The work started with understanding the scheduling requirements and memory hierarchy, followed by implementing the Round Robin scheduler, cache subsystem, and their integration into a single simulator.

Once the core functionality was completed, different test cases were executed to verify correctness and observe cache behavior under various workloads. This step-by-step approach made the implementation process easier to manage and helped identify issues before combining all components into the final solution.

The overall development workflow followed during the project is shown below.

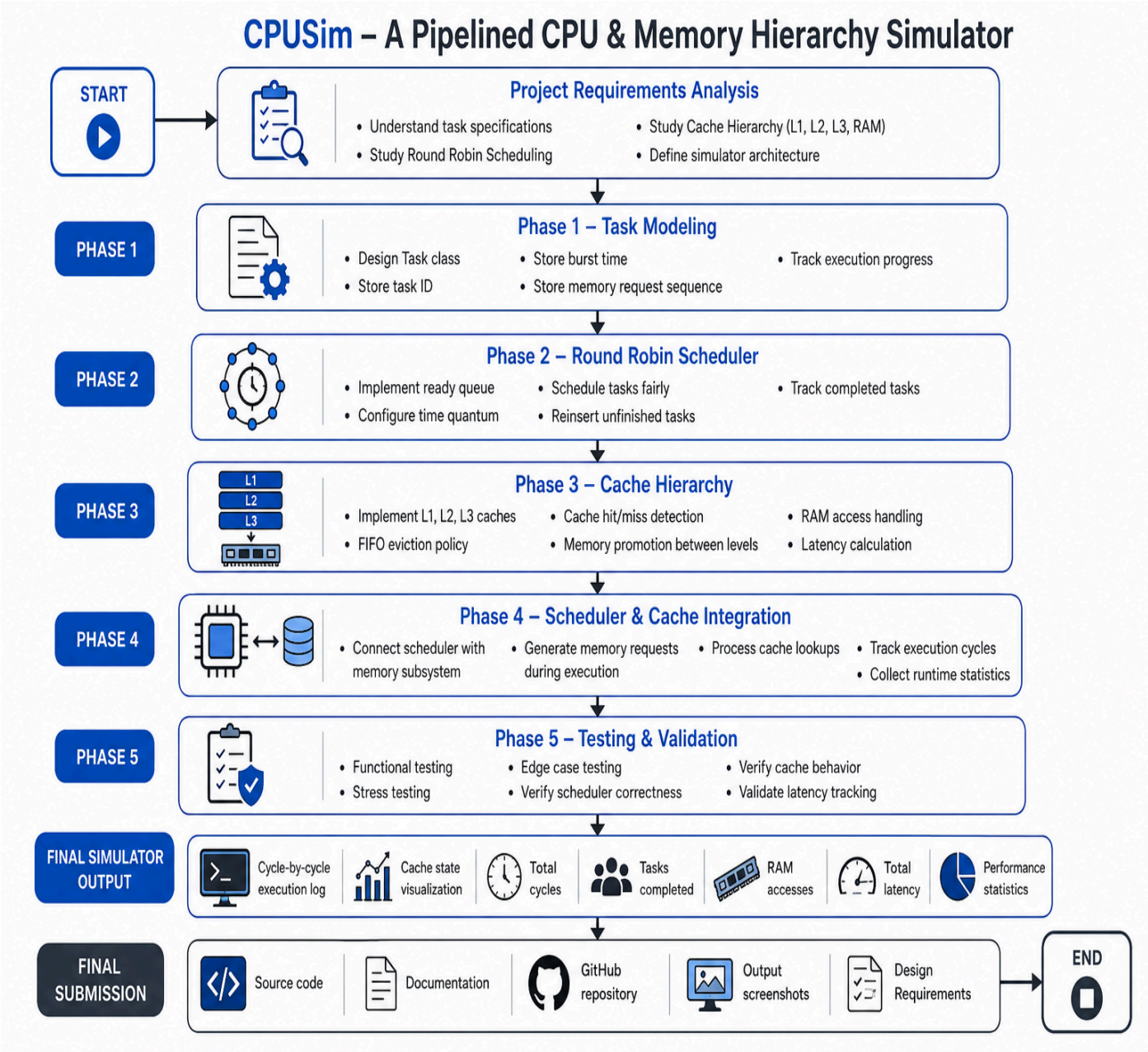


Figure 1: Phase-wise implementation approach followed during development.

2. Problem Understanding

2.1 Task Requirements

The project consists of two major components:

CPU Scheduler

- Reads tasks from the input file

- Decides which task runs at each step
- Tracks task execution and completion

Cache Hierarchy Simulator

- Simulates L1, L2, L3 caches and RAM
- Performs cache hit/miss detection
- Applies FIFO eviction when a cache becomes full
- Promotes memory blocks to higher cache levels
- Tracks RAM accesses and memory latency

Required Output

- Running task at each cycle
- Requested memory block
- Cache state and hit/miss information
- Final execution statistics

2.2 Assumptions

The following assumptions were made before implementation:

- 1 burst unit is treated as 1 CPU cycle
- All cache levels start empty
- Each task performs 1 memory request per CPU cycle
- Memory requests repeat cyclically if burst time is larger than the memory block list
- RAM always contains all memory blocks
- Blocks fetched from RAM or lower cache levels are promoted directly to L1

2.3 Design Goals

The simulator was designed with the following goals:

- Correct scheduling behaviour
- Accurate cache simulation
- Clear cycle-by-cycle visualization
- Modular and maintainable code structure
- Easy support for future extensions such as LRU, multi-core execution, and additional scheduling algorithms

3. Phase 1 – System Design and Planning

Objective

Before starting implementation, I first analyzed the task requirements, identified missing details, and defined a clear architecture for the simulator. The goal of this phase was to understand how the scheduler, cache hierarchy, and memory requests interact before writing any code.

3.1 Requirement Analysis

The simulator consists of two major subsystems:

- CPU Scheduler
- Cache Hierarchy Simulator

The scheduler is responsible for deciding which task gets CPU time, while the cache hierarchy is responsible for locating memory blocks and tracking cache behavior. The simulator must also provide cycle-by-cycle execution details and final execution statistics.

3.2 Assumption Design

A few assumptions were required because some implementation details were not explicitly specified in the task description.

- One burst unit is treated as one CPU cycle.
- All cache levels start empty.
- Each task performs one memory request per CPU cycle.
- Memory requests repeat cyclically if the burst time exceeds the number of listed memory blocks.
- RAM always contains all memory blocks.
- Memory blocks fetched from RAM or lower cache levels are promoted directly to L1.

These assumptions were documented before implementation to keep the simulator behavior consistent.

3.3 Architecture Design

The simulator follows the flow shown below:

Input File → Task Loader → Scheduler → Running Task → Memory Hierarchy → Statistics → Output

The scheduler selects the currently running task. The running task generates memory requests, which are processed by the cache hierarchy using the lookup order L1 → L2 → L3 → RAM. The results are then recorded by the statistics module and displayed in the terminal output.

To keep the code organized, scheduling logic, cache management, and simulation control are treated as separate components with clearly defined responsibilities.

3.4 Validation Strategy

The simulator will be validated using small test cases before full integration.

Validation includes:

- Input parsing verification
- Scheduler correctness testing
- Cache hit/miss testing
- FIFO eviction testing

Special attention will be given to FIFO validation since the provided sample workload does not generate enough unique memory blocks to fill the L1 cache and trigger evictions.

4. Phase 2 – CPU Scheduler

Objective

The goal of this phase was to design the CPU scheduling subsystem responsible for selecting which task receives CPU time during simulation.

4.1 Input Parsing

The simulator reads task information from a text file. Each task contains a task identifier, burst time, and a list of memory blocks.

Example:

TASK T1 BURST 5 MEM M1 M4 M7

The parser extracts this information and converts it into Task objects which are later used by the scheduler.

4.2 Task Representation

Each task stores both static and dynamic information.

Static information:

- Task ID

- Burst Time
- Memory Blocks

Dynamic information:

- Remaining Burst Time
- Current Memory Index

This allows the simulator to track execution progress without modifying the original task definition.

4.3 Scheduling Algorithm Selection

Round Robin scheduling was selected for this project.

The algorithm allocates CPU time fairly by giving each task a fixed time quantum before switching to the next task. Compared to FCFS, Round Robin produces more balanced execution and provides clearer visualization of scheduling behaviour during simulation.

4.4 Scheduler Design

The scheduler maintains a ready queue containing all tasks waiting for CPU time.

During execution:

- A task is selected from the front of the queue.
- The task executes for at most one quantum.
- Remaining burst time is updated.
- Completed tasks are removed.
- Incomplete tasks are placed at the back of the queue.

This process continues until all tasks have finished execution.

4.5 Validation Strategy

Scheduler behaviour will be verified using small test cases with known execution orders.

Validation focuses on:

- Correct task switching
- Proper quantum handling
- Accurate burst time updates
- Correct task completion behaviour
- Fair queue management

5. Phase 3 – Cache Hierarchy

Objective

The goal of this phase was to design the memory hierarchy subsystem responsible for memory lookup, cache hits and misses, block promotion, latency tracking, and FIFO-based eviction.

5.1 Cache Architecture

The simulator models a four-level memory hierarchy consisting of L1, L2, L3 caches and RAM.

Level	Capacity	Latency
L1	32 slots	4 cycles
L2	128 slots	12 cycles
L3	512 slots	40 cycles
RAM	Unlimited	200 cycles

The hierarchy is searched from L1 to RAM. Since higher cache levels have lower latency, accessing data from L1 is significantly faster than accessing data from RAM.

5.2 FIFO Eviction Policy

Each cache level is implemented as a fixed-capacity FIFO queue.

When a cache becomes full and a new block must be inserted, the oldest block is removed before inserting the new block. This follows the First-In First-Out (FIFO) replacement policy specified in the task statement.

Example:

[M1, M2, M3]

Insert M4

[M2, M3, M4]

Evicted Block: M1

5.3 Cache Lookup Flow

Whenever a task requests a memory block, the simulator performs a sequential lookup:

L1 → L2 → L3 → RAM

The search stops immediately after the first successful match.

- If found in L1, an L1 hit occurs.
- If found in L2, an L2 hit occurs.
- If found in L3, an L3 hit occurs.
- If not found in any cache level, the block is fetched from RAM.

The latency associated with the level where the block is found is added to the simulation statistics.

5.4 Memory Promotion Strategy

To improve future access performance, memory blocks are promoted to L1 whenever they are accessed from a lower level.

Promotion occurs in the following cases:

- L2 hit → promote to L1
- L3 hit → promote to L1
- RAM fetch → promote to L1

This allows frequently accessed blocks to move closer to the CPU and reduces future memory access latency.

5.5 Cache Validation

Cache behaviour will be validated using targeted test cases.

Validation includes:

- L1 hit testing
- L2 hit testing
- RAM access testing
- FIFO eviction testing

Additional custom test cases will be created to verify FIFO behaviour because the provided sample workload does not contain enough unique memory blocks to fill the L1 cache and trigger evictions.

6. Phase 4 – Scheduler and Cache Integration

Objective

The goal of this phase was to combine the CPU scheduler and cache hierarchy into a single simulator capable of executing tasks, processing memory requests, tracking latency, and collecting execution statistics.

6.1 Execution Flow

The simulator follows a cycle-based execution model.

For each cycle:

- The scheduler selects the currently running task.
- The task generates a memory request.
- The cache hierarchy processes the request using the lookup order L1 → L2 → L3 → RAM.
- Cache statistics and latency information are updated.
- Execution proceeds to the next cycle until all tasks complete.

This flow integrates scheduling and memory simulation into a single execution pipeline.

6.2 Memory Requests During Execution

Each running task generates one memory request per CPU cycle.

If a task executes for more cycles than the number of memory blocks listed in its definition, the memory block sequence repeats cyclically.

Example:

Memory Blocks: [M1, M4, M7]

Execution:

- Cycle 1 → M1
- Cycle 2 → M4
- Cycle 3 → M7
- Cycle 4 → M1
- Cycle 5 → M4

This approach allows tasks with long burst times to continue generating memory accesses throughout execution.

6.3 Latency Tracking

Every memory access contributes latency depending on the level where the requested block is found.

- L1 Hit → 4 cycles
- L2 Hit → 12 cycles
- L3 Hit → 40 cycles
- RAM Access → 200 cycles

The simulator accumulates this latency throughout execution to analyze memory system performance.

6.4 Statistics Collection

The simulator maintains execution statistics during runtime.

Tracked statistics include:

- Total Cycles
- Tasks Completed
- RAM Accesses
- Scheduler Used

Additional statistics such as cache hits, cache misses, and total memory latency may also be collected for analysis.

6.5 Integration Validation

Integration testing verifies that all components operate correctly when combined.

Validation focuses on:

- Correct scheduler behaviour
- Correct cache lookup behaviour
- Proper memory request generation
- Accurate latency accumulation
- Correct statistics collection
- Successful completion of all tasks

Both sample workloads and custom test cases will be used to validate the complete simulator.

7. Phase 5 – Testing and Analysis

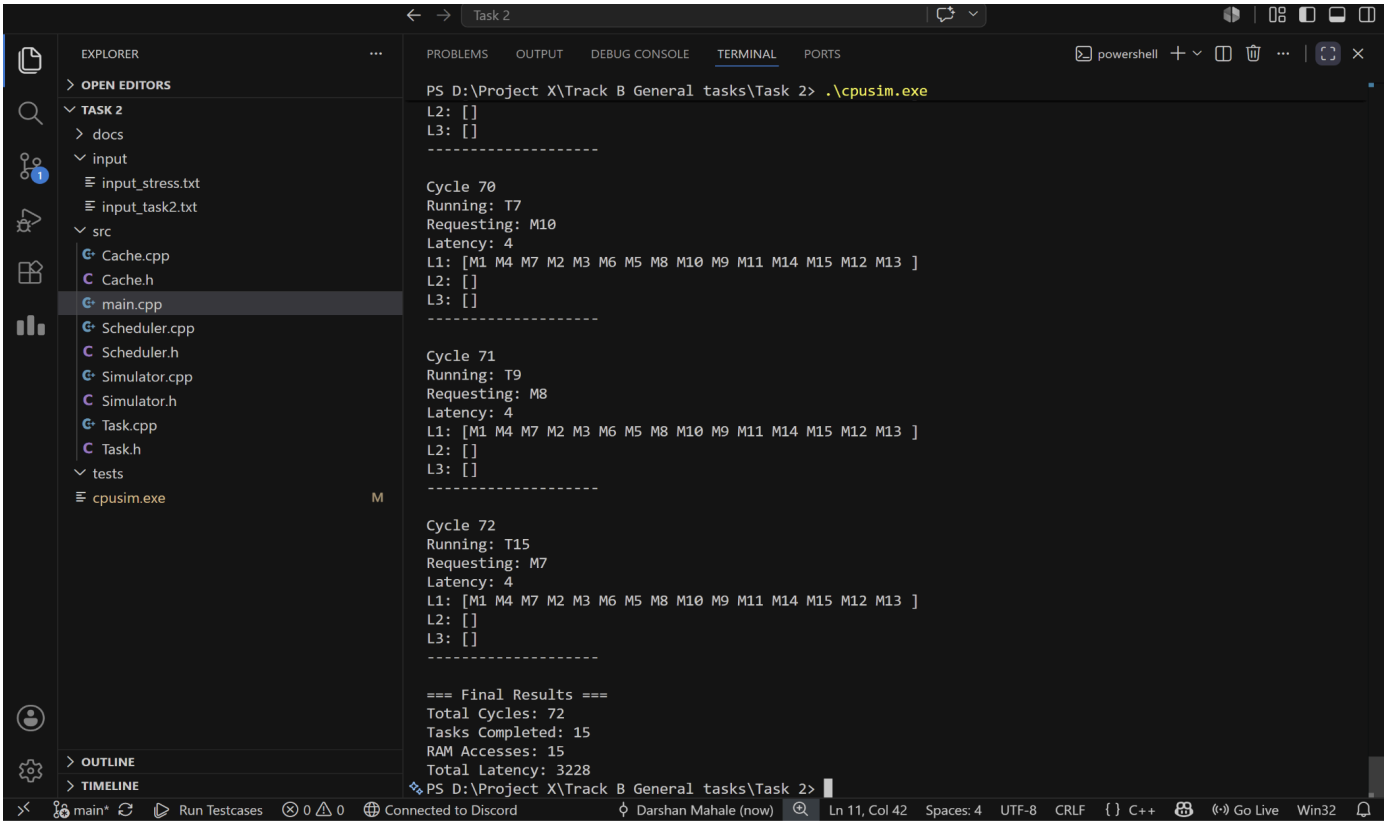
7.1 Functional Testing

The simulator was tested using the provided input file to verify correctness of task scheduling, memory request generation, cache lookup, and statistics collection.

The following aspects were verified:

- Correct parsing of task information from the input file.
- Correct Round Robin scheduling behaviour.
- Correct generation of memory requests during execution.
- Correct latency calculation based on cache hits and misses.
- Correct reporting of final statistics.

Screenshot 1: Normal Simulator Execution



The output demonstrates successful execution of all tasks along with cycle-by-cycle scheduling decisions, memory requests, cache accesses, and final statistics.

7.2 Stress Testing

A custom stress test was created to evaluate cache behaviour when the number of unique memory blocks exceeded cache capacity.

Test characteristics:

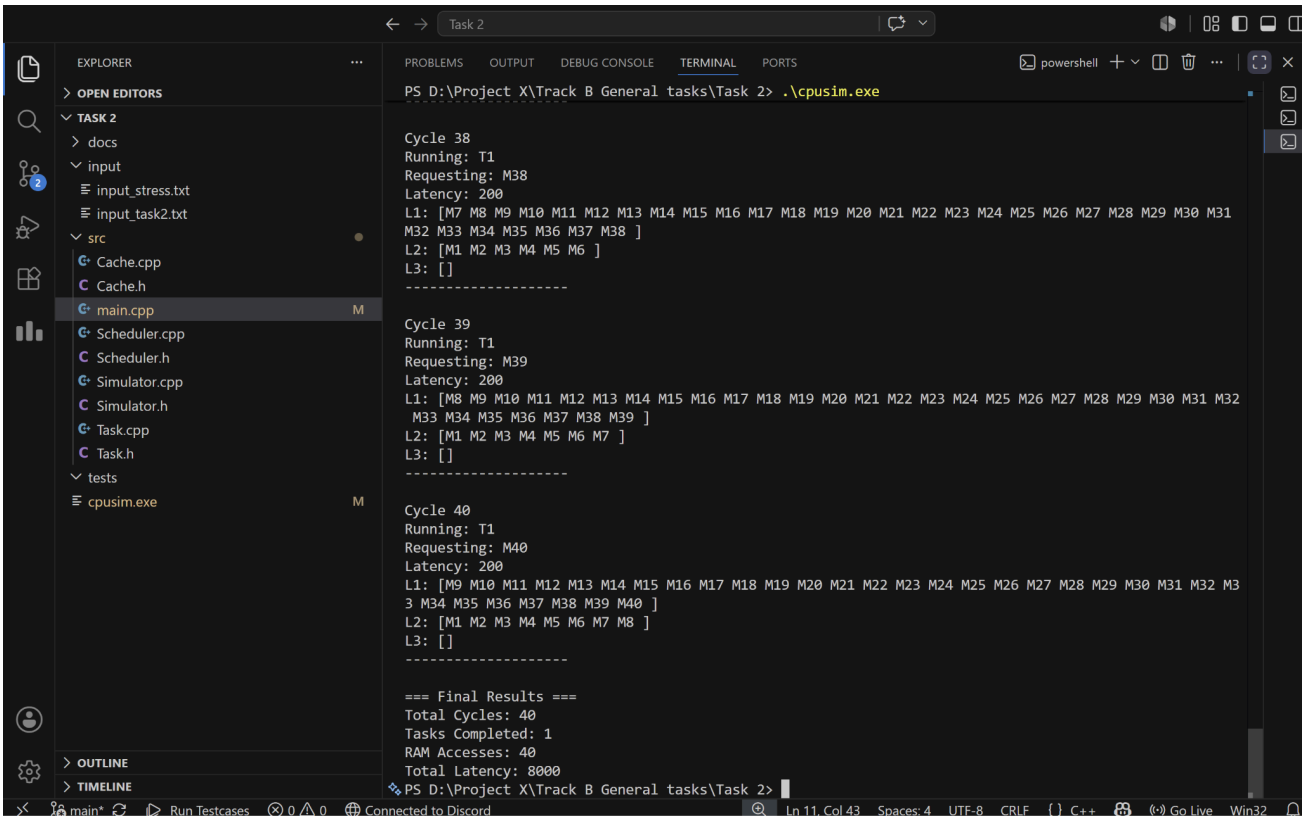
- 1 task
- 40 unique memory blocks
- L1 cache capacity intentionally exceeded

- Continuous memory requests generated

Expected behaviour:

- FIFO eviction should occur.
- Older blocks should be removed from L1.
- Evicted blocks should move to L2.
- RAM accesses should increase due to cache misses.

Screenshot 2: Stress Test Execution



The output confirms that FIFO eviction occurs correctly and cache hierarchy behaviour remains consistent under heavy memory pressure.

7.3 Edge Case Testing

Several edge cases were considered during validation:

- Single task execution.
- Empty cache startup state.
- Repeated access to the same memory block.
- Cache hit scenarios.
- Cache miss scenarios.
- Task completion handling.

All tested cases produced expected behaviour.

7.4 Observations

The following observations were made during testing:

- Cache hits significantly reduced access latency.
- Repeated memory requests benefited from cached data.
- FIFO eviction operated consistently when cache capacity was exceeded.
- Round Robin scheduling distributed CPU time fairly among active tasks.
- The simulator maintained correct statistics throughout execution.

7.5 Limitations

Current implementation limitations:

- Only Round Robin scheduling is implemented.
- FIFO is the only cache replacement policy.
- Single-core CPU simulation.
- No write-back cache policy.
- Cache capacities are fixed at compile time.

8. Implementation Summary

File	Purpose
Task.h / Task.cpp	Task representation and memory request tracking
Scheduler.h / Scheduler.cpp	Round Robin scheduler implementation
Cache.h / Cache.cpp	Multi-level cache hierarchy and FIFO eviction
Simulator.h / Simulator.cpp	Integration of scheduler and cache subsystems
main.cpp	Input parsing and simulator execution

9. Conclusion

A CPU scheduling and cache hierarchy simulator was successfully designed and implemented in C++. The simulator combines Round Robin task scheduling with a multi-level cache system capable of handling cache hits, misses, latency tracking, and FIFO-based eviction.

Testing demonstrated correct behaviour under normal workloads, edge cases, and stress conditions. The final implementation provides a modular and extensible foundation for exploring operating system scheduling and memory management concepts.